



PADRÕES DE PROJETO CRIACIONAIS

O Padrão Prototype

Clonagem de objetos sem acoplamento
e com máxima eficiência.

Aluna: Ana Laura Barbieri Jum

O custo oculto de criar objetos do zero



Construção Complexa

Criar objetos e inicializar atributos repetidamente a partir do zero exige tempo e alto poder de processamento.



A Barreira da Cópia Externa

Copiar um objeto "pelo lado de fora" é frequentemente impossível: campos privados e dependências ocultas impedem a cópia perfeita.

Você não deveria depender de classes concretas apenas para duplicar algo que já existe.

A lógica da autoclonagem

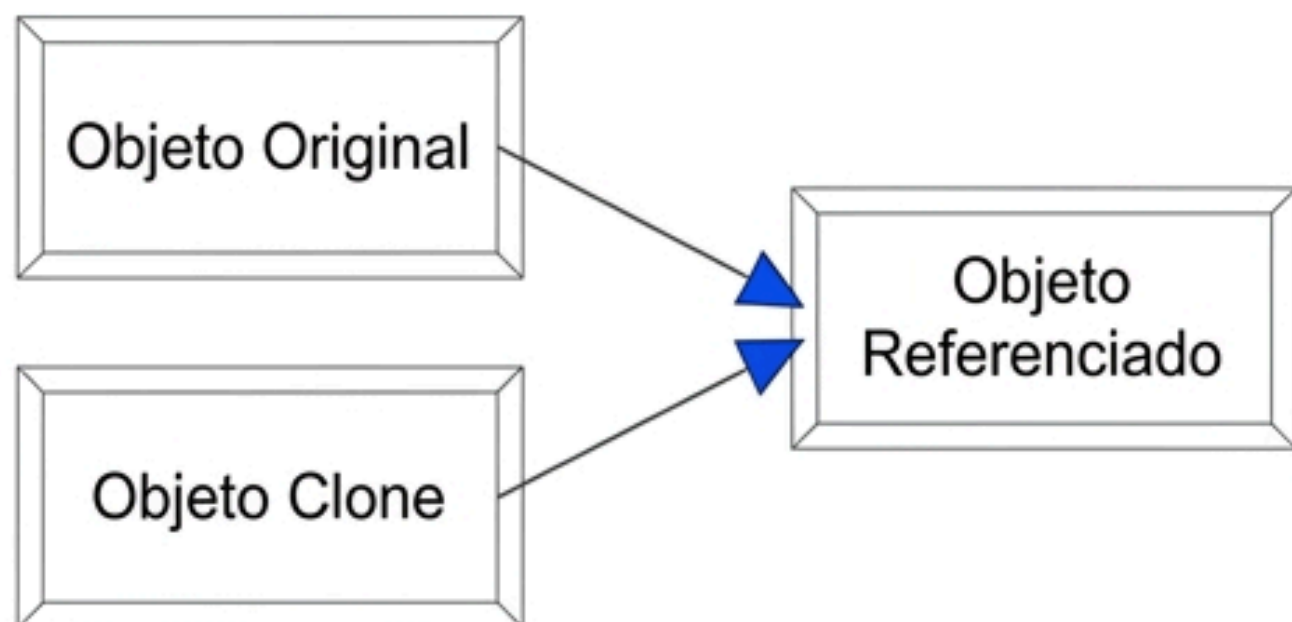
Um padrão de **projeto criacional** que permite copiar objetos existentes sem tornar seu código dependente de suas classes.



Em vez de chamar construtores diretos, o Cliente usa uma interface comum.
O próprio objeto assume a responsabilidade de gerar sua **cópia exata**.

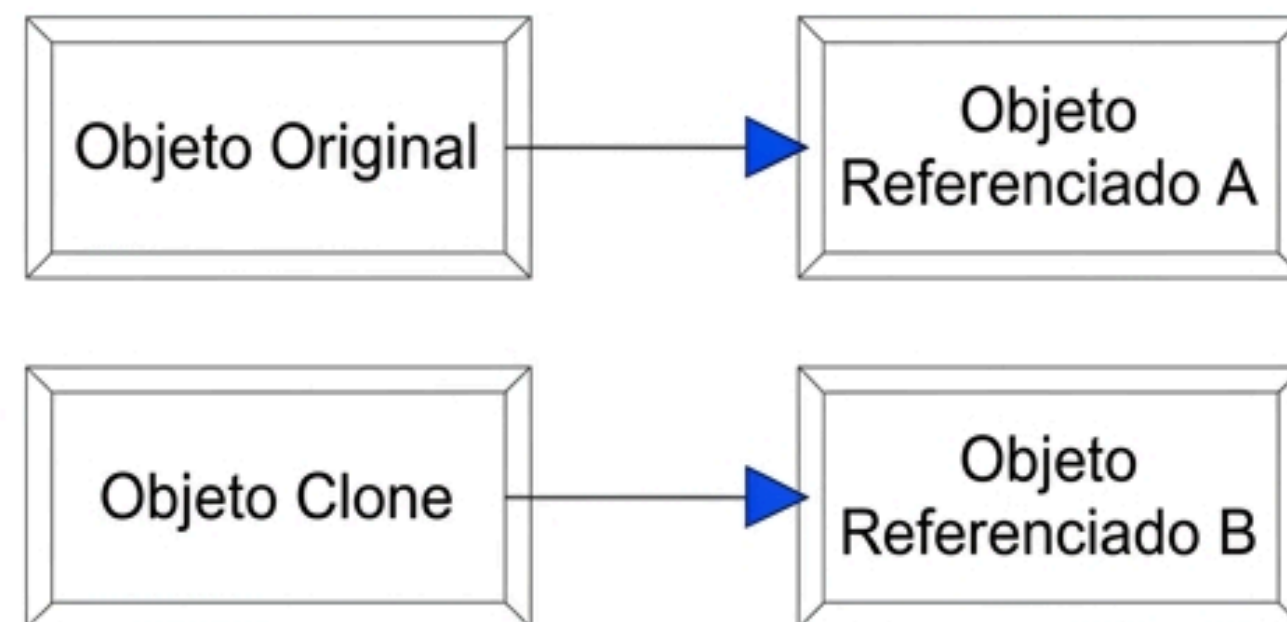
Anatomia da cópia: Rasa versus Profunda

Cópia Rasa (Shallow Copy)



Apenas as referências internas são copiadas.
Os objetos referenciados originais são compartilhados entre o clone e o original.

Cópia Profunda (Deep Copy)



Todos os objetos internos são clonados recursivamente. Independência total da instância original.

Cenários ideais para o uso do Prototype

1

Independência de Classes Concretas

Ideal quando o código recebe objetos de terceiros via interfaces. Não é necessário saber a classe exata por trás da interface para cloná-la.

2

Interface de Clonagem Universal

Um contrato padronizado (`clone()`) permite interagir com qualquer objeto que suporte duplicação de forma transparente ao cliente.

3

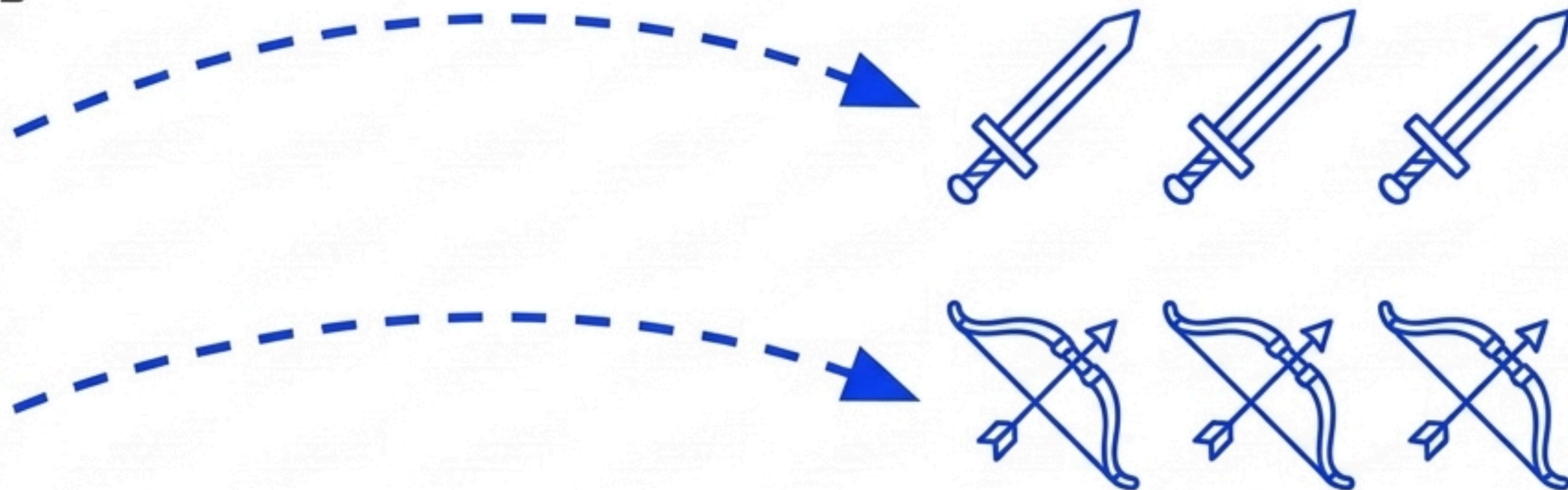
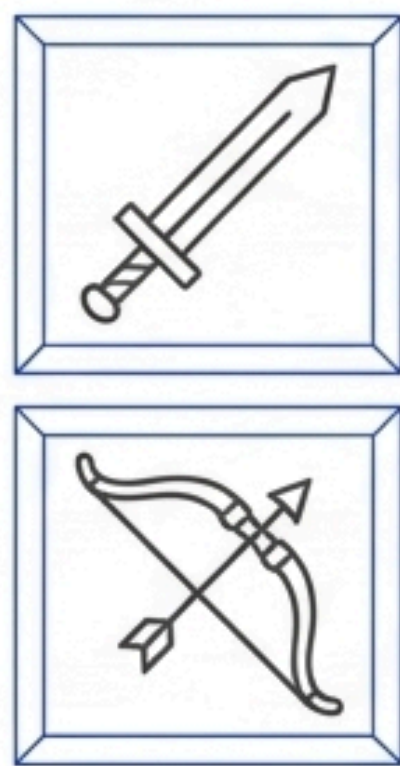
Redução Drástica de Subclasses

Evita a proliferação de classes criadas apenas para inicializar objetos com pequenas variações de configuração pré-determinada.

O caso dos personagens de RPG

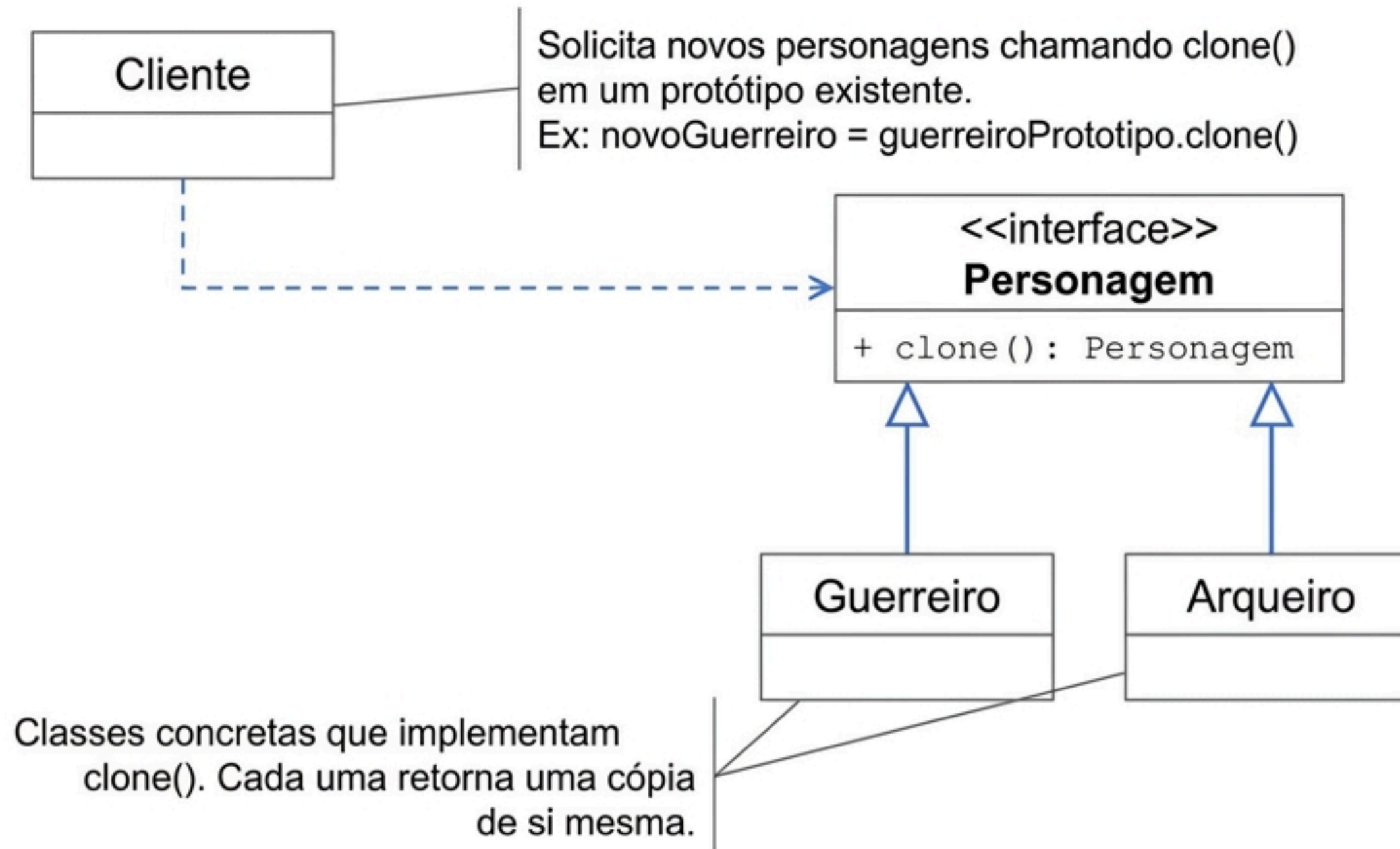
"Em vez de recriar cada personagem do zero toda vez que forem necessários, mantemos instâncias protótipo."

Protótipos Originais



O sistema simplesmente clona o modelo existente. Isso economiza tempo de processamento e garante que todas as instâncias da classe compartilhem as mesmas características iniciais até serem modificadas pelo jogador.

A arquitetura do padrão na prática

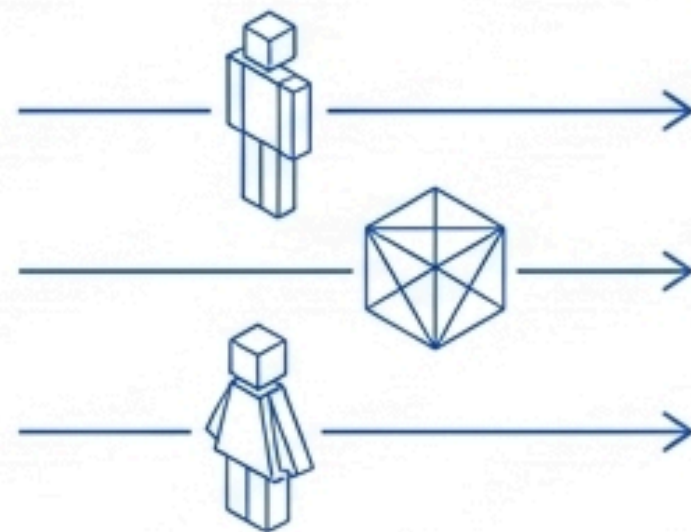


O cliente ignora as classes concretas. Ele apenas **pede uma** cópia do protótipo desejado através da interface genérica.

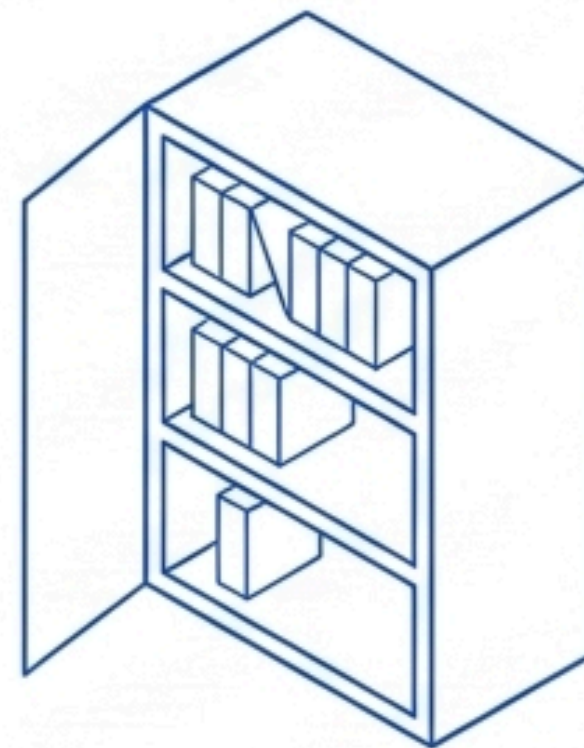
O catálogo central de instâncias

O papel do Registro de Protótipos (Opcional)

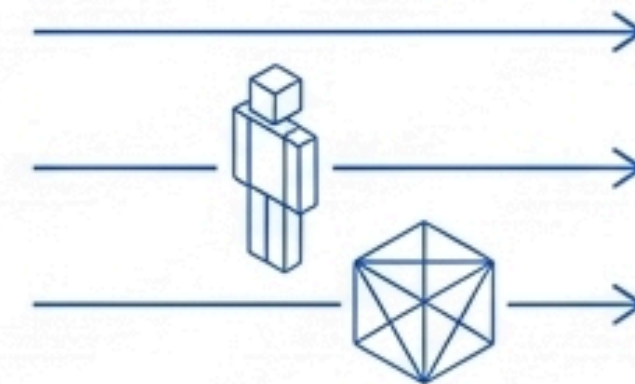
criar(tipo, prototipo): Adiciona um novo modelo pré-configurado ao catálogo.



Prototipo_Personagem



obter(tipo): Localiza o modelo no catálogo, invoca automaticamente o clone() e devolve a cópia perfeitamente instanciada ao Cliente.



Cliente

Este padrão substitui totalmente as chamadas diretas a new por chamadas dinâmicas e limpas ao registro.

Implementação em quatro etapas

01

A Interface. Declarar o método genérico `clone()` em uma interface comum a todos os objetos que serão duplicados.

02

O Construtor. Cada classe concreta cria um construtor que recebe um objeto da mesma classe, copiando seus valores (se for subclasse, aciona a superclasse).

03

A Sobrescrita. Implementar o método `clone()` de forma simples, geralmente chamando o construtor clonador: `return new Classe(this);`

```
return new Classe(this);
```

04

O Catálogo. (Opcional) Criar o Registro central para armazenar as instâncias pré-configuradas mais frequentes.

O impacto direto na arquitetura do código

SEM PROTOTYPE	COM PROTOTYPE
✗ Não existe interface clone().	✓ Classes implementam a interface clone().
✗ Criação manual pesada utilizando new.	✓ O sistema recebe apenas cópias já prontas.
✗ Código Altamente Acoplado: O sistema depende das classes concretas e de seus atributos internos.	✓ Código Totalmente Desacoplado: O sistema gerador ignora os detalhes internos das classes copiadas.
✗ Alta Manutenção: Adicionar um novo personagem exige alterar constantemente o código criador.	✓ Fácil Expansão: Adicionar uma nova classe exige apenas a implementação do seu próprio clone().

Referências

SHALLOWAY, Alan; TROTT, James R. **Design Patterns Explained. [S.l.]: Pearson Education, 2004.**

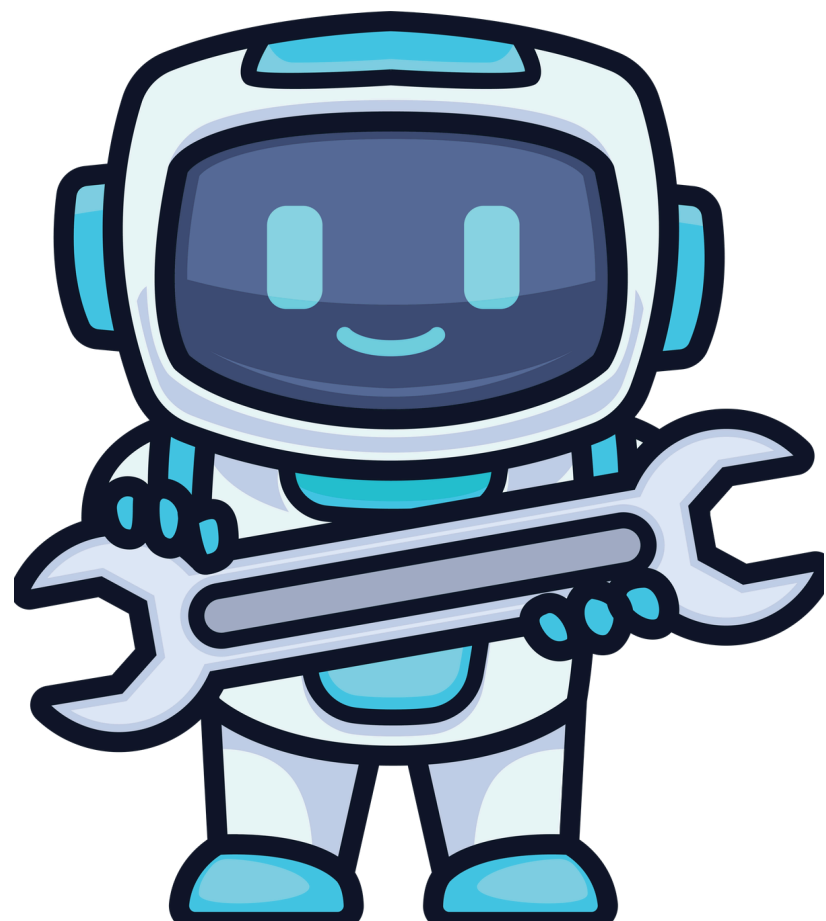
Disponível em:

<https://www.google.com.br/books/edition/Design_Patterns_Explained/84Whswqz4rAC?hl=pt-BR&gbpv=0>. Acesso em set. 2026

Prototype. Disponível em: <<https://refactoring.guru/pt-br/design-patterns/prototype>>. Acesso em: set. 2026.

Pinto, Osiel Pereira. **Padrões de Projeto Aplicados.** Disponível em:

<https://www.google.com.br/books/edition/Padr%C3%B5es_de_Projeto_Aplicados/UHzKEQAAQBAJ?hl=pt-BR&gbpv=0>. Acesso em: set. 2026.



Obrigada!